

Race-Condition / Deadlock Test Strategy

Race-Condition / Deadlock Test Strategy

Revision: 2 Last modified: 2026-07-04T12:00:00Z

Rev 2 (2026-07-04): independently re-verified against SPECIFICATION.md / transport-selection-ladder.md during a corpus-wide gap-analysis pass; the loom-vs-tsan/-race determinism distinction (§6) and the G4-decides-the-canonical- tool framing (QA-D1) are consistent with benchmarking.md's G4 treatment. No contradictions found.

Master technical specification — Volume 8 (Testing & QA), nano-detail document **race-deadlock**, one of the seven §11.4.169 cross-cutting test-type deep-dives. It deepens 10-testing-acceptance-and-qa.md §5.11 (RACE) into an implementation-ready bank for **data-race** and **deadlock** detection on HelixVPN's hot paths, using Rust loom + ThreadSanitizer and Go -race. SPEC-ONLY: it describes the harness, fixtures, evidence, gate, and the paired §1.1 mutation; it does not build the product. Sources cited inline by id — [OVERVIEW] = doc 10; [ladder] = [../v02-data-plane/transport-selection-ladder.md]; [reconcile] = [../v03-control-plane/reconciliation-flow.md]; [TM] = [../v05-security/threat-model.md]; [01-DP] = doc 01. Claims not grounded in the evidence base are marked **UNVERIFIED** per constitution §11.4.6 — never fabricated. (Companion: logical correctness under concurrency is owned by [concurrency.md]; this doc owns *memory-model* races and *lock-order* deadlocks.)

Table of contents

- 0. Scope on HelixVPN surfaces
- 1. The hot paths under interleaving
- 2. Harness — loom + tsan + go -race

- [3. The lock-order discipline \(no blocking op inside a held lock\)](#)
 - [4. Fixtures — real critical sections \(§11.4.27\)](#)
 - [5. Captured evidence \(§11.4.69\)](#)
 - [6. Determinism \(§11.4.50\)](#)
 - [7. Acceptance gate](#)
 - [8. The paired §1.1 mutation](#)
 - [9. Test skeletons](#)
 - [Sources verified](#)
-

0. Scope on HelixVPN surfaces

A data race or deadlock on a VPN's data plane is a §11.4.145-class *latent* defect: it passes code review and every functional test, then fails at runtime only — a torn read of AllowedIPs can **route a packet outside its grant** (a default-deny breach, [TM T-EDGE-E-1]); a deadlock in the reconcile loop stalls convergence past the < 1 s SLO. This bank exercises the memory-model behaviour of HelixVPN's lock-bearing hot paths and asserts a **zero-findings** detector report — a -race, loom, or tsan finding is a release blocker.

Hot path	Concurrent access	Failure if racy	Component
edge verdict-map hot-swap	policy update swaps the map while packets are classified	use-after-free / half-applied map ⇒ wrong verdict	helix-edge (Rust, ArcSwap)
client reconcile loop	network-map apply under a concurrent status read	torn read of AllowedIPs ⇒ packet outside grant	helix-core (Rust)
core tick() keepalive/rekey	timer fires under concurrent send/recv	rekey race ⇒ dropped/duplicated handshake	helix-core (Rust)
transport ladder re-pick	re-pick under concurrent dial result	torn ladder state ⇒ stuck transport	helix-core (ladder.rs, [ladder §3])
coordinator topology graph	concurrent stream fan-out under a new enroll	racy graph read ⇒ wrong served map	helix-go (Go)
Redis Streams consumer	concurrent ack/claim of a pending message	double-process / lost ack	helix-go (Go)

1. The hot paths under interleaving

The bank's value is exhaustively (Rust) or detector-driven (Go) interleaving the *real* critical sections, not asserting "we used a mutex". Two are load-bearing:

- **Edge verdict-map hot-swap.** The edge classifies packets against a VerdictMap while a policy update swaps it. The contract: the swap is **atomic** (ArcSwap store) so a classifier either sees the whole old map or the whole new map — **never a half-applied map** (no torn read, no use-after-free). A loom test models the swap-vs-classify interleaving exhaustively and asserts `is_consistent()` on every observed interleaving (overview §5.11).
- **Client reconcile vs status read.** The reconcile loop rewrites AllowedIPs while a status reader reads the current peer set (for the UI / FFI status stream). The contract: the reader sees a consistent snapshot — never a peer with half-updated AllowedIPs that would route a packet outside its grant. A loom test exhaustively interleaves apply-vs-read.

For the Go control plane, exhaustive interleaving is infeasible, so the discipline is **detector- driven**: every coordinator + consumer test runs under `go test -race` (ThreadSanitizer-based), and a positive deadlock-probe test asserts the fan-out never deadlocks under concurrent enroll+stream.

2. Harness — loom + tsan + go -race

Layer	Tool	What it proves
Rust core/ edge critical sections	<code>cargo +nightly test --features loom</code>	exhaustive interleaving of the modelled section — no torn read, no deadlock, no UAF
Rust FFI boundary	ThreadSanitizer (<code>-Zsanitizer=thread</code>)	data races across the Dart↔Rust FFI boundary the loom model cannot reach
Go coordinator + consumer	<code>go test -race</code>	data races + a subset of lock-order issues on the real Go runtime
Go deadlock probe	a bounded-timeout fan-out test	the fan-out completes within budget (no deadlock) under concurrent enroll+stream

loom is the canonical strong layer (decision QA-D1, overview §10): its exhaustive interleaving is stronger than -race's observe-what-happened for the verdict-swap hot path; QA-D1 revisits this on the G4 Rust-vs-Go edge decision (if the edge ships in Go, the canonical edge tool becomes -race). The bank runs under make test RACE stage (overview §9), dispatched by helix_qa. make test already runs go test -race ./helix-go/... (overview §9), so the Go race detector is on by default.

3. The lock-order discipline (no blocking op inside a held lock)

CLAUDE.md MANDATORY DEVELOPMENT PRINCIPLE 2 — *no blocking operations inside synchronized / shared-lock regions* — is asserted mechanically, not by review. Two checks:

- **loom no-block-under-lock.** A loom test asserts the verdict-swap never holds the classify lock across an allocation or a .await/blocking call (overview §5.11): a long op inside the held lock is the deadlock/timeout vector principle 2 forbids. The test models the section and asserts the lock is released before any potentially-blocking call.
 - **lock-order graph.** A static + runtime lock-ordering check (the Go coordinator acquires locks in a fixed total order: topology-graph lock → per-stream lock, never the reverse) — a loom/-race deadlock-probe asserts no two goroutines acquire the two locks in opposite order (the classic AB-BA deadlock). A documented lock-acquisition order is part of the spec; a violation is a finding.
-

4. Fixtures — real critical sections (§11.4.27)

Race testing is below the unit layer in the sense that mocks cannot prove a memory-model property: the fixtures are the **real** critical-section types compiled with the detector.

Fixture	What	Why real
real VerdictMap + ArcSwap	the actual edge verdict-map type under loom	a mocked map cannot reproduce the real swap interleaving
real reconcile AllowedIPs apply	the actual client reconcile critical section	a stubbed apply cannot exhibit the real torn-read

Fixture	What	Why real
real coordinator topology graph	the actual Go graph + per-stream locks	-race must instrument the real lock acquisition order
loom model harness	loom's permutation driver wrapping the real types	loom needs the real Arc/atomic types, not a fake

Note (§11.4.27): loom substitutes its own `loom::sync/loom::thread` primitives for `std` ones — this is the *model* substitution loom requires, not a product mock; the *logic under test* is the real critical section.

5. Captured evidence (§11.4.69)

Every PASS cites the detector report under `qa-results/race/<run-id>/`:

Test	Artifact	Asserts
RACE-VERDICT-SWAP	<code>loom_verdict_swap.log</code>	every interleaving consistent; zero UAF; zero deadlock
RACE-RECONCILE-READ	<code>loom_reconcile.log</code>	status read sees a consistent AllowedIPs snapshot
RACE-FFI-TSAN	<code>tsan_ffl.log</code>	zero ThreadSanitizer findings across the FFI boundary
RACE-GO-CONSUMER	<code>go_race_consumer.log</code>	go test -race clean (zero data-race reports)
DEADLOCK-FANOUT	<code>fanout_timing.json</code>	fan-out completes within budget under concurrent enroll+stream (no deadlock)
NOBLOCK-UNDER-LOCK	<code>loom_noblock.log</code>	classify lock released before any blocking call (principle 2)

The detector's **zero-findings** report is the captured evidence — a clean `loom/tsan/-race` log is the §11.4.69 artifact. A "no error in the test run" claim without the attached detector log is a B2 absence-of-error bluff (overview §0).

6. Determinism (§11.4.50)

loom's exhaustive interleaving is **deterministic by construction** — it explores the full modelled state space, so a clean loom run is a proof over all interleavings (within the model's bound), not a sample; re-running yields the identical exploration. tsan and go -race are probabilistic (they observe only the interleavings that happened), so the bank runs them under `ab_run_n_times ... 10` (N=10 cycle-validation, because races are high-risk per §11.4.132) and the evidence-hash is over (`findings == 0: bool`); all 10 MUST report zero findings. A single run with a finding in 10 is **auto-FAIL** — a race that appears once in ten is still a release blocker (§11.4.50: intermittent is mechanically forbidden, and a race is the *definition* of intermittent). Where loom is bounded (`LOOM_MAX_PREEMPTIONS`), the bound is recorded in the evidence so the proof's scope is honest (§11.4.6).

7. Acceptance gate

Gate	Bar	Evidence	Component
RACE-VERDICT-SWAP	loom: zero inconsistent interleaving / UAF / deadlock	loom_verdict_swap.log	helix-edge
RACE-RECONCILE-READ	loom: status read consistent	loom_reconcile.log	helix-core
RACE-FFI-TSAN	tsan: zero findings	tsan_ffi.log	FFI
RACE-GO-CONSUMER	go test -race clean	go_race_consumer.log	helix-go
DEADLOCK-FANOUT	fan-out within budget, no deadlock	fanout_timing.json	helix-go
NOBLOCK-UNDER-LOCK	classify lock released before blocking call	loom_noblock.log	helix-edge

A -race/loom/tsan finding is a **release blocker** (overview §5.11) — it is the §11.4.145 angle-3 latent-bug class that causes runtime-only failures. RACE cells appear in the coverage ledger for F-DEFAULT-DENY, F-TRANSPORT-ESCALATE, F-POLICY-

RECONCILE, F-KILLSWITCH (overview §6.3). The race floor runs alongside the §11.4.132 risk-ordered security floor (the verdict-swap race *is* a default-deny breach vector).

8. The paired §1.1 mutation

MUTATION (paired §1.1, gate CM-VERDICT-SWAP-ATOMIC):

Replace the ArcSwap atomic store of the VerdictMap with a non-atomic

two-step update (clear-then-populate), so a classifier can observe a

half-applied (empty/partial) map mid-swap.

EXPECTED: the loom test observes an interleaving where `is_consistent()`

is false (a packet classified against a half-applied map) →

RACE-VERDICT-SWAP FAILs → mutation caught (loom is exhaustive,

so it is GUARANTEED to find the interleaving, not probabilistic).

RESTORE: re-instate the ArcSwap atomic store; re-run → GREEN.

A second mutation (CM-GO-RACE-CLEAN) introduces an unguarded concurrent write to the coordinator's topology graph; expected: `go test -race` reports a data race → RACE-GO-CONSUMER FAILs → caught. A third (CM-NOBLOCK-UNDER-LOCK) inserts a blocking call inside the held classify lock; expected: the loom no-block assertion FAILs (and the deadlock-probe may time out) → caught. All mutations restored, tree verified quiescent (§11.4.84) before commit.

9. Test skeletons

```
// helix-edge/tests/verdict_swap_loom.rs - RACE-VERDICT-SWAP: no
// torn read, no deadlock (overview §5.11)
#[test] fn verdict_swap_is_atomic_under_loom() {
    loom::model(|| {
        let vm =
            Arc::new(ArcSwap::from_pointee(VerdictMap::empty()));
        let (vm1, vm2) = (vm.clone(), vm.clone());
        let t = loom::thread::spawn(move || {
            vm1.store(Arc::new(VerdictMap::allow("10.10.0.0/24"))); //
            // policy update swaps the map
        });
        let v = vm2.load(); // classifier
        // reads concurrently
```

```

    assert!(v.is_consistent());           // NEVER a half-
    applied map (no torn read / UAF)
    t.join().unwrap();
});
// paired §1.1 (CM-VERDICT-SWAP-ATOMIC): non-atomic two-step
// store → loom finds an inconsistent read → FAIL
}

// helix-edge/tests/noblock_under_lock_loom.rs — NOBLOCK-UNDER-
// LOCK (principle 2)
#[test] fn classify_lock_released_before_blocking_call() {
    loom::model(|| {
        let guard = CLASSIFY_LOCK.lock();
        let verdict = classify_fast(&guard);    // pure, non-
        blocking, under the lock
        drop(guard);                          // lock MUST be
        released BEFORE any blocking work

        do_possibly_blocking_followup(verdict); // alloc / await
        happens OUTSIDE the lock
        assert!(lock_was_released_before_followup());
    });
}

// helix-go/internal/coordinator/fanout_deadlock_test.go —
// DEADLOCK-FANOUT (run with -race)
func TestFanoutNoDeadlockUnderConcurrentEnroll(t *testing.T) {
    infra := mustBootRedisPG(t)
    coord := startCoordinator(t, infra)
    done := make(chan struct{})
    go func() { openNStreams(coord, 50) }()    //
    concurrent stream fan-out
    go func() { enrollNDevices(coord, 50); close(done) }() //
    concurrent enroll (mutates topology graph)
    select {
    case <-done:                                //
        completed → no deadlock
        writeEvidence(t, "qa-results/race/fanout_timing.json",
            coord.Timing())
    case <-time.After(deadlockBudget):
        t.Fatal("DEADLOCK: fan-out did not complete within budget
            (AB-BA lock order?)")
    }
    // run: go test -race → CM-GO-RACE-CLEAN mutation (unguarded
    // graph write) makes -race report a race → FAIL
}

```

Honest boundary (§11.4.6). loom proves the property over the **modelled** state space within its preemption bound — it does **not** prove freedom from races in code paths not wrapped in a loom model (those rely on tsan/-race, which are probabilistic and proven only over N=10 observed runs, not exhaustively). The G4 edge-language decision (overview §7.1) determines whether loom (Rust) or -race (Go) is the canonical edge tool — until G4 resolves,

the canonical edge RACE tool is **UNVERIFIED** (QA-D1 recommends Rust/loom but defers to the G4 CSV). The Go deadlock-probe proves *no deadlock under the tested concurrency level*, not under all loads — higher-concurrency deadlock exposure is an §11.4.118 discovery concern.

Sources verified

- [OVERVIEW] [../10-testing-acceptance-and-qa.md] — §5.11 (RACE strategy + loom verdict-swap skeleton + lock discipline), §6.3 (RACE ledger cells), §7.1 (G4 edge language), §8 (determinism/risk-order), §9 (go test -race in make test), §10 QA-D1. (Read 2026-06-26.)
- [ladder] [../v02-data-plane/transport-selection-ladder.md] — §3 ladder state machine (re-pick under concurrent dial result). (Read 2026-06-26.)
- [reconcile] [../v03-control-plane/reconciliation-flow.md] — §0.2 reconcile loop, coordinator topology graph + stream fan-out. (Read 2026-06-26.)
- [TM] [../v05-security/threat-model.md] — T-EDGE-E-1 (a torn verdict map is a default-deny breach vector). (Cited via overview.)
- [01-DP] final/01-data-plane.md — helix-core tick() keepalive/rekey, edge verdict map. (Cited via overview.)
- Constitution: §11.4.169, §11.4.27 (no-fakes / loom model is not a product mock), §11.4.69 (detector report = evidence), §11.4.50 (determinism: race is the definition of intermittent), §11.4.132 (race is high-risk), §11.4.145 (latent-runtime-only-defect class), §11.4.84 (quiescence), §1.1 (paired mutation); CLAUDE.md principles 2 + 3.